

Introduction to OS

From baremetal loops to kernel-mode syscalls: how an operating system earns its keep between the hardware and everything running on top of it — and what has to be true for that OS to also be *real-time*.

Level — embedded / systems Reference arch — ARMv8-A (AArch64) Reference kernel — Linux

§1 Introduction to OS

An operating system is the software layer that turns raw hardware — a CPU, some RAM, a handful of peripherals — into something applications can actually use without every program having to reinvent device drivers, memory management, and task switching from scratch.

Strip away the terminology and an OS is doing three jobs at once:

- **Resource manager** — arbitrates CPU time, memory, and I/O devices between multiple competing programs.
- **Abstraction layer** — gives applications a stable, hardware-independent interface (files, sockets, processes) instead of forcing every program to speak directly to silicon.
- **Protection boundary** — keeps one program's bugs (or malice) from corrupting another program, or the kernel itself.

Everything else in this handout — user/kernel mode, system calls, scheduling policy, real-time guarantees — is really just different angles on how an OS delivers those three jobs, and what it costs to do so.

§2 Baremetal C vs OS

A baremetal program *is* the whole system — there's no kernel underneath it arbitrating anything. An OS-hosted program is one tenant among many, mediated by a kernel it can never fully see or touch directly. The practical differences fall out of that one distinction:

ASPECT	BAREMETAL C	OS-HOSTED (LINUX-LIKE)
Execution model	<code>main()</code> super-loop plus interrupt handlers	Many processes/threads, time-multiplexed by a scheduler
Hardware access	Direct memory-mapped register writes	Only via system calls into kernel-mode drivers
Memory model	Flat address space; addresses = physical addresses	Virtual memory per process, enforced by the MMU
Determinism	Fully deterministic — you account for every cycle	Scheduling jitter; deterministic only if the OS is an RTOS
Startup	You supply the vector table, linker script, and reset handler	Bootloader loads a kernel, kernel starts an init process
Fault isolation	One bug can take down the entire system	A crashing process need not affect others
Footprint	Kilobytes	Megabytes (kernel + libc + userspace)

WHERE THIS SITS IN YOUR OWN WORKFLOW

Every libopencm3 + FreeRTOS project you build is baremetal C plus a cooperating scheduler bolted directly onto it — there's no MMU-enforced isolation, no syscalls, no separate address spaces. FreeRTOS gives you multitasking without paying for the rest of what "OS" usually implies. That's a deliberate trade, not a missing feature — it's what makes worst-case latency boundable at all (more on this in [§7](#)).

§3 Single processor and multiprocessor systems

Single-processor (uniprocessor) systems have exactly one core executing one instruction stream at a time. Any "concurrency" you observe — multiple FreeRTOS tasks appearing to run at once — is time-slicing: the illusion of parallelism, not the fact of it.

Multiprocessor systems have two or more CPUs sharing memory, and split into two flavors:

- **SMP (symmetric multiprocessing)** — every core is identical and runs the same OS image; the scheduler can place any task on any core. This is what "multicore" means on a modern application processor (e.g. a quad-core Cortex-A53 running one Linux kernel across all four cores).
- **AMP (asymmetric multiprocessing)** — cores are different, or run *different* software stacks entirely, each with its own role. A Xilinx Zynq-7000 is the textbook example: dual Cortex-A9 cores capable of running Linux, sitting on the same die as FPGA fabric — the "processing system" and "programmable logic" are asymmetric partners, not interchangeable peers.

Worth noting for the RTOS side of this handout: vanilla FreeRTOS assumes a single scheduler instance managing one core (uniprocessor, even if the silicon has other cores doing something else entirely, AMP-style). FreeRTOS-SMP extends this to symmetric multicore microcontrollers, but the classic model — and everything in your STM32 build system — is single-core.

§4 Types of systems

These four aren't competing designs so much as successive answers to the same question — "how do we keep an expensive CPU busy and give users a tolerable response time?" — asked again as hardware got cheaper and expectations rose.

~1950s	Early 1960s	1961–1965	1970s–80s
Batch processing	Multiprogramming	Time-sharing	Interactive systems
IBM 1401/7094 era. Jobs queued on tape, run unattended, output printed later.	OS/360 keeps several jobs resident so the CPU never idles waiting on I/O.	MIT CTSS, then Multics — many terminals, one CPU, rapid switching.	UNIX shells, then GUIs (Xerox Alto, Macintosh) — immediate feedback becomes the default.

a. Essential features of batch processing

- Jobs are collected ("batched") and run without any user interaction during execution.
- A **resident monitor** automatically sequences one job after another — no operator intervention between jobs.
- **Spooling** moves job input/output between slow media (cards, tape) and faster disk buffering, decoupling I/O speed from CPU speed.
- Optimized for **throughput**, not response time — turnaround was measured in hours, sometimes overnight.
- Jobs carried their resource requirements in a **job control language** (JCL) read by the monitor.

IBM 1401/7094

punch cards

magnetic tape

JCL

b. Time-sharing systems

The CPU is sliced into short time quanta and rapidly rotated across multiple interactive terminal users, each of whom experiences the illusion of having the machine to themselves. This is the direct ancestor of every multi-user and multitasking system since.

- MIT's **CTSS** (1961) is generally credited as the first working time-sharing system.
- **Multics** (1965, MIT/Bell Labs/GE) took the idea further and directly influenced the design of **UNIX** a few years later.
- Response time, not just throughput, becomes a first-class design goal.

c. Multiprogramming

The enabling idea underneath time-sharing: keep *several* jobs resident in memory at once, so when the currently-running job blocks on I/O, the CPU switches to a different job that's ready to run, instead of sitting idle. IBM's **OS/360** (mid-1960s, MFT/MVT variants) brought this to mainframe

workloads. Multiprogramming maximizes CPU utilization; time-sharing adds the additional constraint that switching has to happen fast enough to feel responsive to a human at a terminal.

d. Interactive systems

A system is interactive when a user's input produces an (apparently) immediate response — command in, output back, in a tight feedback loop — rather than a job submitted and collected later. Interactivity rides on top of time-sharing and multiprogramming foundations: terminals replacing punch cards through the 1960s–70s made it possible, UNIX shells made it normal, and graphical desktops (Xerox Alto, then the Macintosh in the 1980s) made it the default expectation that persists today, down to the shell or debugger session you're using on your own STM32 workspace.

§5 User mode and kernel mode – Linux on ARMv8

ARMv8-A defines four **Exception Levels** (ELs) — a strict privilege hierarchy the hardware itself enforces, not just an OS convention:

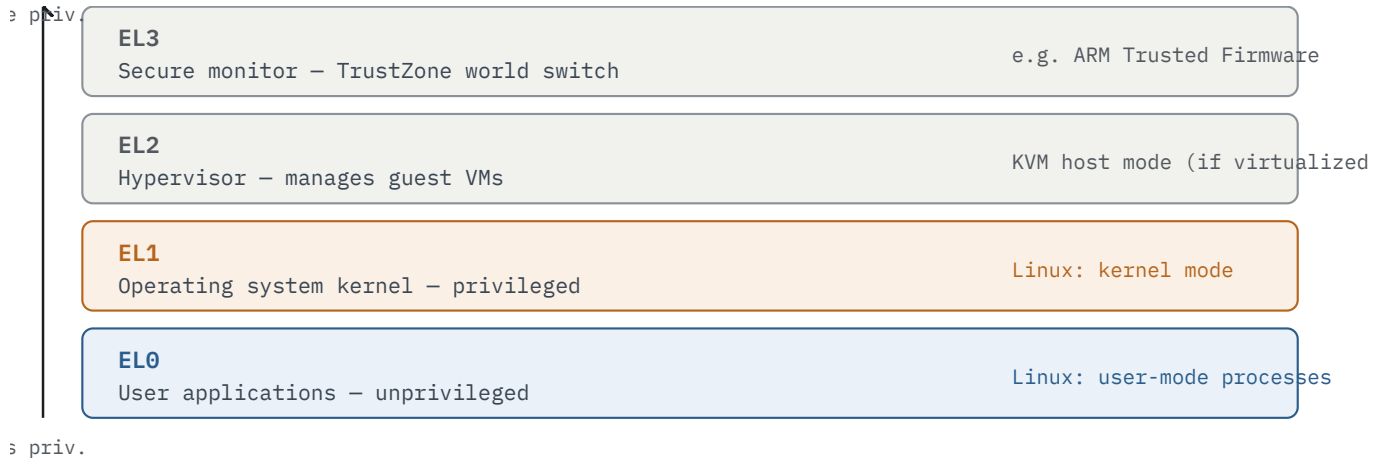


Fig. 1 – A native (non-virtualized) Linux install only ever uses EL0 and EL1; EL2/EL3 come into play with virtualization and TrustZone.

On a typical Linux system with no hypervisor involved, only the bottom two levels matter day to day:

EL0 · USER MODE

- Every normal process runs here — your shell, your applications, glibc itself.
- Cannot execute privileged instructions or touch hardware registers directly.
- Uses its own translation tables via `TTBR0_EL1` — one virtual address space per process.
- Uses stack pointer `SP_EL0`.

EL1 · KERNEL MODE

- The Linux kernel itself — schedulers, drivers, the VFS, network stack.
- Full access to hardware, MMU configuration, and privileged system registers.
- Kernel address space mapped via `TTBR1_EL1`, shared across all processes.
- Uses its own dedicated stack pointer, `SP_EL1` — one per task, allocated from kernel memory.

The transition between them is always triggered by an **exception** — there is no other legal path from EL0 to EL1. Exceptions are either:

- **Synchronous** — caused directly by the instruction stream: an explicit `SVC` (system call), or a fault such as a page fault or undefined instruction.

- **Asynchronous** — IRQ, FIQ, or SError, arriving independently of what the CPU happened to be executing.

On any exception into EL1, the hardware — not software — automatically performs a fixed sequence: save the return address into `ELR_EL1` , save the processor state into `SPSR_EL1` , switch the active stack pointer to `SP_EL1` , and jump into the exception vector table pointed to by `VBAR_EL1` . Which of the table's 16 entries gets used depends on the exception type *and* whether it came from the current EL or a lower one — the system call path (§6) uses one specific entry of that table.

§6 Implementation of a system call – Linux, ARM64

Take `write(fd, buf, count)` as the running example — everything below is the real path for any arm64 Linux syscall, just with a different handler function at the end.

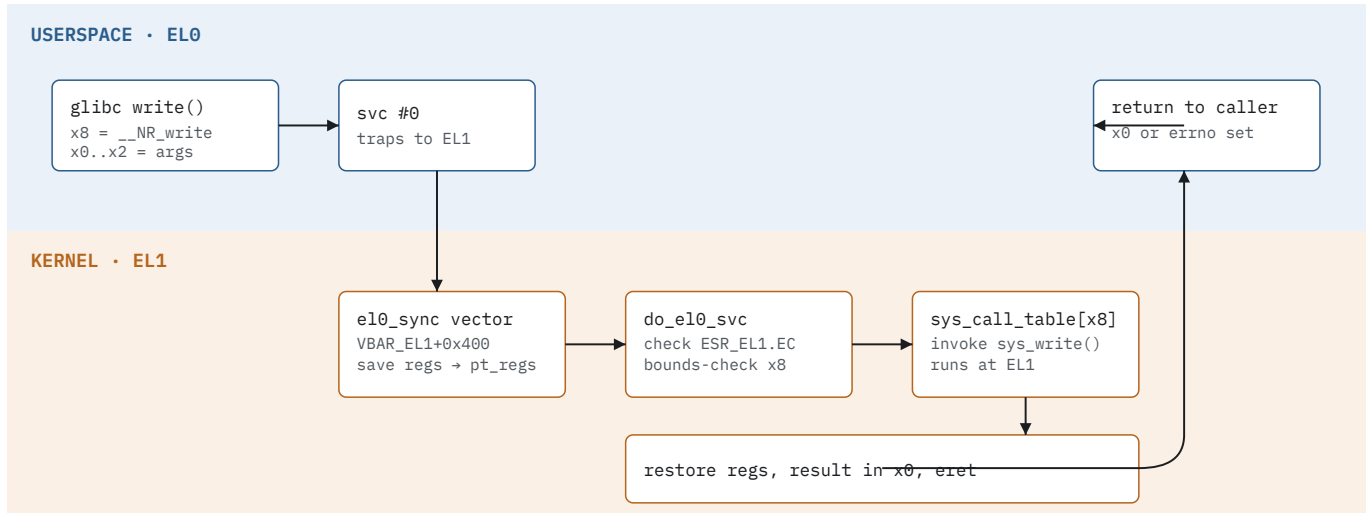


Fig. 2 – One round trip: user → SVC trap → kernel dispatch → handler → ERET → user.

- User-side setup.** The glibc wrapper places the syscall number in `x8` (arm64's convention — 32-bit ARM historically used `r7` instead) and up to six arguments in `x0 – x5`, then executes `svc #0`. The immediate operand is unused by Linux; it's always `0`.
- Hardware trap.** `svc` raises a synchronous exception. The CPU atomically: copies the return address into `ELR_EL1`, copies PSTATE into `SPSR_EL1`, switches the active stack pointer to `SP_EL1` (the current task's kernel stack), and jumps to `VBAR_EL1 + 0x400` — the vector table slot for "synchronous exception from a lower EL, using AArch64."
- Kernel entry.** The `el0_sync` handler in `arch/arm64/kernel/entry.S` saves every general-purpose register plus `SP_EL0`, `ELR_EL1`, and `SPSR_EL1` onto that kernel stack, building a `struct pt_regs` — a complete snapshot of user-space's state at the moment of the trap.
- Classify the exception.** The kernel reads `ESR_EL1` (Exception Syndrome Register); its `EC` field is `0x15` for an AArch64 `SVC`. This is what distinguishes "this was a system call" from a page fault or undefined instruction landing at the very same vector entry.
- Dispatch.** `do_el0_svc` reads the syscall number back out of `pt_regs->regs[8]`, bounds-checks it against `__NR_syscalls`, and indexes `sys_call_table[]` — an array of function pointers built at kernel-compile time from the syscall table definitions.
- Handler runs at EL1.** `sys_write(fd, buf, count)` executes with full kernel privilege. Since `buf` is a user-space pointer, the handler can't just dereference it — it goes through `copy_from_user()` / `copy_to_user()`, which validate the address actually lies within the calling process's own `TTBR0_EL1` mapping before touching it.

- 7 **Return value.** Whatever the handler returns is written back into `pt_regs->regs[0]` (i.e. what will become `x0`). By POSIX convention, glibc later turns a small negative range of return values into `errno` plus a `-1` return, rather than passing the raw negative number through.
- 8 **Return to user.** The kernel restores every register from `pt_regs` and executes `eret`. This single instruction atomically restores `PC` from `ELR_EL1` and `PSTATE` from `SPSR_EL1`, dropping the CPU back to EL0 at the instruction immediately after the original `svc`.

```
// conceptually, arch/arm64/kernel/entry.S + entry-common.c
el0_sync:
    kernel_entry 0                // save x0-x30, SP_EL0, ELR_EL1, SPSR_EL1 -> pt_regs
    mrs    x1, esr_el1            // read exception syndrome
    lsr    x24, x1, #26           // extract EC field
    cmp    x24, #ESR_ELx_EC_SVC64 // 0x15 == SVC from AArch64
    b.eq   el0_svc

el0_svc:
    bl     do_el0_svc             // C: bounds-check x8, call sys_call_table[x8]
    b      ret_to_user

ret_to_user:
    kernel_exit 0                // restore pt_regs -> registers
    eret                        // PC <- ELR_EL1, PSTATE <- SPSR_EL1, drop to EL0
```

WHY THIS MATTERS EVEN OUTSIDE LINUX

This exact EL0→EL1→EL0 round trip — trap, save state, dispatch by table, restore state, atomic return — is the same shape as an interrupt or a FreeRTOS context switch on a Cortex-M, just without exception levels (Cortex-M has no EL0/EL1 split; privilege is a single bit). Understanding one gives you the other almost for free.

§7 What makes an OS a RTOS

The defining property of a real-time OS is **bounded worst-case latency**, not raw speed. A system that always responds within 5 ms is more "real-time" than one that averages 50 μ s but occasionally stalls for 200 ms. RTOS design is entirely organized around making that worst case knowable and small:

- **Priority-based preemptive scheduling** as the default policy — the highest-priority ready task always runs next, rather than the fairness/throughput goals a general-purpose scheduler (like Linux's CFS) optimizes for.
- **Bounded critical sections.** Kernel operations — taking a lock, handling a tick — must complete in small, constant, analyzable time. An RTOS kernel can't contain an internal operation that might occasionally take an unbounded amount of time.
- **Priority-inversion protocols** built into synchronization primitives — priority inheritance or priority ceiling — so a low-priority task holding a shared resource can't block a high-priority task indefinitely.
- **Predictable interrupt latency** — the time from "interrupt asserted" to "handler running" has a known upper bound.
- **Deterministic memory behavior** — many RTOS kernels avoid a general-purpose heap (or provide fixed-size pool allocators) because searching a fragmented heap isn't time-bounded.

THE MARS PATHFINDER STORY

In 1997 the Pathfinder rover, running VxWorks, began resetting itself intermittently. The cause was classic unbounded priority inversion: a low-priority meteorological task held a mutex that a high-priority bus-management task needed, while a medium-priority task kept preempting the low-priority one — so the high-priority task waited far longer than its deadline allowed. Engineers fixed it remotely, in flight, by enabling priority inheritance on the mutex. It's the textbook illustration of why "priority inversion protocol" is a design requirement, not an optimization.

Hard vs soft real-time

Hard real-time — missing a deadline is a system failure (airbag deployment timing, engine ignition control). **Soft real-time** — missing a deadline degrades quality but isn't catastrophic (a late video frame, a delayed sensor sample). Whether you need an RTOS at all, and how strict a one, follows directly from which category your deadlines fall into.

Worth noting: a general-purpose OS isn't automatically disqualified. Mainline Linux's `PREEMPT_RT` patch set makes the kernel almost fully preemptible and applies priority inheritance to kernel locks, pushing stock Linux much closer to soft (and in some configurations, near-hard) real-time behavior — without changing the syscall path described in §6 at all.

§8 Real-time embedded systems

What an RTOS actually buys you is easiest to see against real hardware/software pairings, spanning very different stakes:

DOMAIN	HARDWARE	RTOS	WHY REAL-TIME MATTERS HERE
Hobbyist / drones	STM32F4/F7 (Cortex-M4/M7)	FreeRTOS	Flight-controller loops (attitude estimation, motor mixing) must run on a fixed period or the craft becomes unstable.
Industrial control	NXP i.MX RT crossover MCU	FreeRTOS / Zephyr	Motor-drive control loops need microsecond-level, jitter-free timing.
Automotive safety	TI Hercules / Infineon AURIX (Cortex-R, lockstep)	AUTOSAR OS / SafeRTOS	ABS and airbag control are hard real-time <i>and</i> safety-certified (ISO 26262); lockstep cores catch silicon-level faults.
Industrial / robotics	Xilinx Zynq-7000 (dual Cortex-A9 + FPGA)	PREEMPT_RT Linux or Xenomai on the A9s; FPGA fabric for hard-	Combines a Linux-capable core for orchestration with FPGA determinism for the control

DOMAIN	HARDWARE	RTOS	WHY REAL-TIME MATTERS HERE
		deadline logic	loop that can't tolerate any jitter at all.
Aerospace	RAD-hardened PowerPC / ARM	VxWorks	Mars rovers (Pathfinder, Curiosity, Perseverance) — see the priority-inversion story above.
Automotive infotainment	ARM Cortex-A application processors	QNX Neutrino	Microkernel fault isolation plus real-time guarantees for the instrument cluster, even while the infotainment side runs untrusted apps.
IoT / consumer	ESP32 (dual-core Xtensa)	FreeRTOS	One core often runs the Wi-Fi/Bluetooth stack, the other application logic — FreeRTOS is what makes that split manageable.

